The Journal of Open Source Software

annotater: Enhancing library load calls in R

Luis D. Verde Arregoitia [1] **and Juan Cruz Rodríguez** [2]

1 Laboratorio de Macroecología Evolutiva, Red de Biología Evolutiva, Instituto de Ecología, A.C., Carretera Antigua a Coatepec 351, Col. El Haya, Xalapa, 91073, Veracruz, Mexico **2** FAMAF, Universidad Nacional de Córdoba, Argentina

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 06 May 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Extensions and packages extend the capabilities of a programming language, and working with source code and scripts rather than interactively lets us document and repeat our workflows. However, in the R ecosystem, the sheer number and diversity of existing packages can be overwhelming. In this context, the purpose of individual packages or their role in projects can become unclear. To address this lack of context, one practical approach is to add information directly to scripts using code comments. Code comments are annotations within code meant for the human reader, not the machine, meant to provide additional information or clarity to what is being executed (Filazzola & Lortie, 2022).

`annotater` is an R package for automated commenting of library load calls in R scripts, or text-based formats that allow for embedded code blocks such as R Markdown and Quarto (`.rmd` and `.qmd` files, respectively).

Statement of need

The functions in `annotater` address an unmet need in R for improving code comprehension regarding loaded packages in scripts. Most scripts load numerous packages, which may not have self-explanatory names and are often loaded without mentioning their purpose, source, or which specific functions and datasets are actually used. This lack of explicit information does not imply bad coding practices, but adding useful information as unobtrusive comments can lead to self-documented and understandable code, ultimately improving individual and collaborative workflows.

When opening a script, the role of a loaded package may not be evident, requiring manual investigation. This might mean interrupting our work to check the documentation or search the web to understand more about the loaded packages. This context switching (Wilson, 2021) can slow down code review, collaboration, and reduce productivity, especially when there are many dependencies or when code is shared between users with different backgrounds and personal 'dialect' preferences (e.g., users of different package 'families' for data manipulation, spatial data work, or statistical modeling frameworks).

In addition, tracking the exact versions and sources of loaded packages is important for ensuring the reproducibility of analyses and results. For example, using the stable vs. development version of a package might mean the difference between a workflow failing or succeeding. Manually noting this information can be tedious or prone to error, but `annotater` functions can easily note the source and version of a package in a user's machine. This approach does not guarantee the automatic recreation of the original execution environment and is not meant to replace existing tools that create comprehensive reproducible environments, such as renv, Docker, or Nix.

Lastly, identifying which parts of a script rely on specific packages and their components can

be challenging, making it harder to refactor code, manage dependencies, or identify unused packages.

Features and examples

Upon installation, R packages already include useful details that we can leverage to automate the creation of these informative comments. These annotations can be particularly useful for sharing code with others, as a way to provide immediate context about why each package is being used and for what purpose. The code in a script can also be examined programatically so that the functions, methods, or datasets being used from each package can also be added as comments.

Code can be annotated interactively using the package functions or through addins in the RStudio IDE.

The following annotations are supported. The code blocks below show the output of the different features on small scripts.

- Add package titles

```
library(brms) # Bayesian Regression Models using 'Stan'
library(caper) # Comparative Analyses of Phylogenetics and Evolution in R
library(readr) # Read Rectangular Text Data
library(picante) # Integrating Phylogenies and Ecology
```

- Add package installation sources and versions. Supports various sources including CRAN, GitHub, GitLab, Bioconductor, Posit Package Manager (RSPM), and R-universe.

```
library(brms)    # [github::paul-buerkner/brms] v2.22.11
library(caper)   # CRAN v1.0.3
library(readr)   # Posit RSPM v2.1.5
library(picante) # CRAN v1.8.2
```

- Identify functions and datasets being used from each package

```
# functions
library(brms) # No used functions found
library(caper) # No used functions found
library(readr) # read_csv
library(picante) # df2vec

dat <- read_csv("mdata.csv")
df2vec(dat, colID = Y1)

# data
library(caper) # shorebird.data
library(readr) # No loaded datasets found
library(picante) # No loaded datasets found

data(shorebird)
hist(shorebird.data$F.Mass)
```

- Compatible with both `library()` and `p_load()` calls when loading packages with `pacman`

```
# add source and version to pacman call
library(readr) # Posit RSPM v2.1.5
pacman::p_load(
caper,         # CRAN v1.0.3
picante        # CRAN v1.8.2
)
```

DRAFT

- Expand popular metapackages into their loaded components. Will change `library(tidyverse)` into:

```
####
library(ggplot2)
library(tibble)
library(tidyr)
library(readr)
library(purrr)
library(dplyr)
library(stringr)
library(forcats)
library(lubridate)
####
```

In its development version, `annotater` supports adding R and RStudio versions, platform, and operating system to the beginning of a script.

Concluding remarks

`annotater` is available on GitHub, CRAN, and R-universe and has a dedicated website for documentation (https://annotater.liomys.mx/). Since its release on CRAN, `annotater` has been downloaded over 12,000 times. Community adoption can be inferred from public code searches for patterns commonly generated by `annotater`. For example, GitHub queries for strings such as "`) # CRAN v`", "`) # Create elegant`", and "`) # A grammar`" -which typically annotate library calls for packages like `dplyr` and `ggplot2`- return over 1,000 results. These matches suggest that a significant number of users are using `annotater` to automatically append version and title comments to their package load calls.

It is worth noting that Large Language Model (LLM) tools can now generate inline explanations for loaded packages. However, `annotater` represents a more parsimonious approach with distinct practical advantages. `annotater` runs locally in R, requiring no internet access, incurring no usage fees, and eliminating the need for setting up local models or managing API keys. Most importantly, package information is obtained directly from users' installations, avoiding issues related to the source and copyright of training data for external LLM tools.

The `annotater` package provides a valuable solution by offering a non-invasive method to automatically add informative comments alongside package load calls. By annotating scripts with package titles, repository sources, versions, and even the functions and datasets being used, `annotater` significantly enhances code clarity and provides essential information for reproducibility and maintenance.

Acknowledgements

We acknowledge the LatinR community for ongoing feedback and promotion of the package.

References

Filazzola, A., & Lortie, C. (2022). A call for clean code to effectively communicate science. *Methods in Ecology and Evolution*, *13*(10), 2119–2128. https://doi.org/https://doi.org/10.1111/2041-210X.13961

Wilson, G. (2021). Task Interruption in Software Development Projects. In *It Will Never Work in Theory*. https://neverworkintheory.org/2021/08/09/task-interruption-in-software-development-proje html

DRAFT